

Superoptimizing from LLVM IR to RISC-V

1 Project Introduction and Description of Work

The idea behind this project is to automatically generate a backend from LLVM IR to RISC-V using enumerative synthesis and superoptimization.

There are two main motivations for this project. The primary motivation is decoupling the development of low-level IRs and ISAs by generating compiler backends from semantics, ensuring both correctness and ease of development, and reducing the cost of adding a new backend to MLIR (a flexible compiler framework for defining IRs at various abstraction levels) by generating a significant portion automatically.

A secondary benefit would be to evaluate the performance of synthesized backends, opening the door to further usage of superoptimization, which should hopefully lead to better performance by incurring a one-time compute cost.

The work would consist of an initial research phase in which I would gather more context on the topic. Here, I would read related papers and leverage my supervisor to build a better understanding of superoptimization and enumerative synthesis. Following this, I will work on defining semantics for a RISC-V dialect which will act as the target of the superoptimization. The focus will be on the arithmetic instructions of the base RISC-V instruction set.

After obtaining the semantics, work will be performed on using a superoptimizer which will initially attempt to generate mappings from LLVM IR instructions to RISC-V instructions. The superoptimizer will generate possible short RISC-V instruction sequences and attempt to find equivalence between the LLVM IR instruction and the generated sequence (using some open-source SMT solver). These equivalences will then be used to generate RISC-V assembly from LLVM IR code.

The superoptimizer will be run on a department computer. The metrics will be recorded on the synthesis time, as well as how much of the LLVM IR instruction set can be lowered successfully.

2 Starting Point

I don't have much additional knowledge beyond what is covered in the tripos. However, I briefly skimmed over some papers suggested by the supervisor on superoptimization and MLIR semantics.

3 Summary of Work

Work will consist of first writing a compilation pass from RISC-V to SMT to define RISC-V semantics. Testing will be performed to ensure the correctness of the defined semantics. After that, the existing superoptimization infrastructure will be used to generate a lowering from LLVM IR to RISC-V by leveraging the defined semantics. The generated lowering will then be evaluated using the methods described below.

4 Success Criteria

- Superoptimize LLVM IR using the arithmetic subset (no memory) of RISC-V with no extensions as the target. Cover most LLVM IR instructions that can be reasonably expressed using 1–3 RISC-V instructions.
- Have reasonable performance when compared against the LLVM backend.
- Ensure correctness of semantics, when tested against RISC-V simulator.

5 Possible Extensions

- Extend supported target ISA with bitwise (B) or multiplication (M) extension for RISC-V by defining semantics for these extensions, enabling lowerings for more LLVM IR operations.
- Add optimizations to prune the synthesis search space.
- Synthesize n-to-m mappings of instructions to increase the effectiveness of the superoptimization step.
- Generate ISA IR from ISA specification to allow generation of a simple ARM (or other ISA) backend.
- Use HPC cluster to attempt to generate more complex mappings, and evaluate performance improvements (or lack thereof).

6 Evaluation Methods

- Evaluate correctness of semantics by running tests and ensuring results are consistent.
- Measure the time it takes to synthesize the lowering per instruction.
- Record the size of the subset of LLVM IR instructions for which the lowering can be synthesized.
- Measure performance on various programs (composed of supported instructions) against the LLVM backend.

7 Estimated Timeline

Weeks 1–2 (16 Oct – 29 Oct)

Goal: A document with a summary of performed research, checked by supervisor to ensure sufficient understanding of the topics.

- Research MLIR, superoptimization, SMT semantics.

Weeks 3–4 (30 Oct – 12 Nov)

Goal: Semantics for a few instructions from the base RISC-V ISA. Tested for correctness against a RISC-V simulator.

- Write a simple optimization pass for existing RISC-V dialect to gain familiarity with it.
- Start work on writing a compilation pass to obtain SMT semantics from the RISC-V dialect, focusing on basic instructions.

Weeks 5–6 (13 Nov – 26 Nov)

Goal: Mostly complete semantics for the arithmetic subset of RISC-V with no extensions, checked for correctness against RISC-V simulator.

- Low-workload buffer space for Michaelmas Term workload.
- Ensure correctness of semantics.
- Finish defining semantics for instructions required to meet success criteria.

Weeks 7–8 (27 Nov – 10 Dec)

Goal: Meet success criteria, working project. Supervisor agrees that performance is reasonable to meet success criteria.

- Connect defined RISC-V SMT semantics to existing MLIR generic superoptimizer.
- Evaluate performance, correctness, and coverage of LLVM IR instructions.

Weeks 9–10 (11 Dec – 24 Dec)

Goal: Met success criteria, working project.

- Low workload buffer for Michaelmas Term revision.
- Ensure success criteria is met.

Weeks 11–12 (25 Dec – 7 Jan)

Goal: Document describing next steps and planned extensions discussed with supervisor.

- Low workload buffer for holidays and spending time with family.
- Extension work.

Weeks 13–14 (8 Jan – 21 Jan)

Goal: Progress report draft handed to supervisor

- Write progress report draft.
- Extension work / Category theory take home test

Weeks 15–16 (22 Jan – 4 Feb)

Goal: Submitted progress report.

- Practice progress report presentation.
- Extension work.

Weeks 17–18 (5 Feb – 18 Feb)

Goal: Finish achievable extensions.

- Extension work.
- Hand mostly complete project to supervisor for feedback.

Weeks 19–20 (19 Feb – 4 Mar)

Goal: Complete project.

- Finish up project work, incorporating feedback.
- Slack space for Lent coursework.

Weeks 21–22 (5 Mar – 18 Mar)

Goal: Introduction and Preparation chapter drafts handed to supervisor.

- Write Introduction and Preparation chapter drafts.

Weeks 23–24 (19 Mar – 1 Apr)

Goal: Implementation chapter draft handed to supervisor.

- Write Implementation chapter draft.
- Revise previous chapters.

Weeks 25–26 (2 Apr – 15 Apr)

Goal: Evaluation and Conclusion chapter drafts handed to supervisor.

- Write Evaluation and Conclusion chapter drafts.
- Revise previous chapters.

Weeks 27–28 (16 Apr – 29 Apr)

Goal: Submitted dissertation.

- Revise and polish dissertation.

Weeks 29–30 (30 Apr – 15 May)

Goal: Safety net for unforeseen delays.

8 Resources Required

I require access to Department computers for superoptimization passes. I will use my personal laptop for coding, as well as writing the dissertation.

I accept full responsibility for this machine and I have made contingency plans to protect myself against hardware and/or software failure.